Questions on Common Loop Algorithms on March 9, 2026

Contents

1 Why was adding an else statement to the bool function "10000%" wrong? 1

2 I get confused on the boolean before the main function 4

3 Can we do this exercise, without making it a function? 8

3.1 Code without a function to return the number of occurrences of a target string in the original string . . . 8

3.2 Code with a function to return the number of occurrences of a target string in the original string . . . 9

4 I wasn't able to understand this problem, even though Ms. Yi wrote the solution. 10

5 I am not fully understanding what I should do for functions when we have to find a value from the length. 12

6 In the old midterms there were a lot of questions about void. Will we have those questions in the midterm? 12

7 Can you give us some practice questions that cover most of the topics we have done so far? 12

8 Where is project 1E? 12

9 Are classes and file input on the midterm? 12

10 What will the midterm exam cover and not cover? 12

11 When will we review for the midterm? 12

12 This wasn't hard but not using namespace std makes it difficult honestly, the code in c++ is extremely difficult. 13

13 What version of C++ are we using? 13

1 Why was adding an else statement to the bool function "10000%" wrong?

Why would it not return the same as if you omitted it? I assumed it was omitted because it's unnecessary.

Answer: No, using the following structure is not just unnecessary, it is wrong.

```
for (...) {
  if (...) {
    return false;
  }
  else {
    return true;
  }
}
```

Here is a code to find out whether an integer is prime or not.

```
//correct version
bool isPrime(int n) {
  if (n <= 1) {
    return false;
  }

  for (int i = 2; i <= n/2; i++)
      {
    if (n % i == 0) {
      //i is a factor of n
      return false;
    }
  }

  return true;
}
```

```
1  //WRONG version
2  bool isPrime(int n) {
3    if (n <= 1) {
4      return false;
5    }

7    for (int i = 2; i <= n/2; i++)
       {
8      if (n % i == 0) {
9        //i is a factor of n
10       return false;
11     }
12     else { //WRONG
13       return true;
14     }
15   }

17   return true;
18 }
```

The leftside code is correct while the rightside one is wrong.

By Line 7-12 in the rightside figure, when i is 2, if `n % i == 0` return false (i.e., not a prime), otherwise, return true.

According to this logic, only 2 is tested, not the rest of non-trivial factor candidates 3, 4, ..., n/2.

For example, suppose n is 9. Since 9 cannot be divided by 2, the code on the right would return `true`. However, even though 9 cannot be divided by 2, it can be divided by 3. Therefore, 3 is a factor of 9 other than 1 and 9 itself, so 9 is **not** a prime number.

Note: I use the boundary $n/2$ for simplicity. A more efficient approach is to use $\sqrt{n}$.

The correct logic is as follows: once we find the **first** non-trivial factor from the candidate pool $[2, n/2]$, we can conclude that n is not a prime and return `false`.

Only after we test **every** non-trivial factor candidate in $[2, n/2]$ and none of them is an actual factor of n should we conclude that n is a prime.

Alternatively, you can use the following code, which employs a `foundNonTrivialFactor` flag to terminate the loop early once a non-trivial factor of n is found.

```
bool isPrime(int n) {
  if (n <= 1) {
    return false;
```

```
  }

  bool foundNonTrivialFactor = false;
  for (int i = 2; i <= n/2 && !foundNonTrivialFactor; i++) {
    if (n % i == 0) { //i is a factor of n
      foundNonTrivialFactor = true;
    }
  }

  return !foundNonTrivialFactor;
  //that is, if foundNonTrivialFactor is true,
  //return false (n is not a prime),
  //otherwise, return true (n is prime).
}
```

A complete code is as follows.

```
//code link: https://onlinegdb.com/FKbJ0kXYr
//purpose:
//1. define isPrime function. Given an integer, find out whether this integer is prime or not.
//2. Call isPrime function to find out all prime integers in [0, 10]
//3. Enter start and end. Call isPrime function to find out the maximum prime int in [start, end].

//sample output
//2
//3
//5
//7
//enter start: 1
//enter end: 1000
//the maximum prime integer in [1, 1000] is 997

//another sample input/output:
//2
//3
//5
//7
//enter start: 20
//enter end: 22
//no prime number in [20, 22]
#include <iostream>
#include <string>
#include <cmath> //sqrt

bool isPrime(int n);

int main() {
```

```
  //print out all prime numbers in [1, 10].
  for (int n = 1; n <= 10; n++) {
    if (isPrime(n)) {
      //condition (isPrime(n)) is the same as (isPrime(n) == true)
      std::cout << n << std::endl;
    }
  }

  //Find out the maximum prime number in [start, end]
  std::cout << "enter start: ";
  int start;
  std::cin >> start;

  std::cout << "enter end: ";
  int end;
  std::cin >> end;

  bool foundPrime = false;
  for (int n = end; n >= start && !foundPrime; n--) {
  //!foundPrime is the same as foundPrime == false
    if (isPrime(n)) {
      std::cout << "the maximum prime integer in ["
        << start << ", " << end << "] is "
        << n << std::endl;

       //find a prime number
       foundPrime = true;
    }
  }

  if (!foundPrime) {
    std::cout << "no prime number in [" << start << ", " << end << "]" << std::endl;
  }
  return 0;
}

bool isPrime(int n) {
  if (n <= 1) {
    return false;
  }

  bool foundNonTrivialFactor = false;
  for (int i = 2; i <= n/2 && !foundNonTrivialFactor; i++) {
    if (n % i == 0) { //i is a factor of n
      foundNonTrivialFactor = true;
    }
```

```
  }

  return !foundNonTrivialFactor;
  //that is, if foundNonTrivialFactor is true,
  //return false (n is not a prime),
  //otherwise, return true (n is prime).
}
```

2 I get confused on the boolean before the main function

You mean why we put the header of user-defined function before `main` function?

In most programming languages, this follows a declare-call-define structure. In languages like C++, you must declare the function (prototype) before main so the compiler knows it exists, then define the logic after main.

Here is how you would do that in C++:

```
//code link: https://onlinegdb.com/epynKylTmZ
//purpose:
//1. define isPrime function. Given an integer, find out whether this integer is prime or not.
//2. Call isPrime function to find out all prime integers in [0, 10]
//3. Enter start and end. Call isPrime function to find out the maximum prime int in [start, end].

//sample output
//2
//3
//5
//7
//enter start: 1
//enter end: 1000
//the maximum prime integer in [1, 1000] is 997

//another sample input/output:
//2
//3
//5
//7
//enter start: 20
//enter end: 22
//no prime number in [20, 22]
#include <iostream>
#include <string>
#include <cmath> //sqrt

bool isPrime(int n);
```

```
int main() {
  //print out all prime numbers in [1, 10].
  for (int n = 1; n <= 10; n++) {
    if (isPrime(n)) {
      //condition (isPrime(n)) is the same as (isPrime(n) == true)
      std::cout << n << std::endl;
    }
  }

  //Find out the maximum prime number in [start, end]
  std::cout << "enter start: ";
  int start;
  std::cin >> start;

  std::cout << "enter end: ";
  int end;
  std::cin >> end;

  bool foundPrime = false;
  for (int n = end; n >= start && !foundPrime; n--) {
  //!foundPrime is the same as foundPrime == false
    if (isPrime(n)) {
      std::cout << "the maximum prime integer in ["
        << start << ", " << end << "] is "
        << n << std::endl;

       //find a prime number
       foundPrime = true;
    }
  }

  if (!foundPrime) {
    std::cout << "no prime number in [" << start << ", " << end << "]" <<
        std::endl;
  }
  return 0;
}

bool isPrime(int n) {
  if (n <= 1) {
    //previously, I wrote condition to be n == 1,
    //which is wrong. It should be n <= 1
    return false;
  }
  //it happens that when n is 2 or 3, it is a prime.
  //But when n is less than 1, n is not prime.
```

```
  //(1) Let 1 and n be trivial factors of n.
  //    Then all other factors of n are non-trivial factors.
  //(2) The possible candidates for non-trivial factors of n
  //    can be as small as 2 and as big as n/2.
  //(3) It remains to test whether there is any ACTUAL non-trivial
  //    factors of n in [2, n/2].
  //(4) If we find out at least one non-trivial factor of n in [2, n/2],
  //    n is not prime.
  //(5) If we cannot find ANY non-trivial factor of n,
  //    that is, the only factors of n is 1 and itself,
  //    then n is prime.
  //only when n/2 >= 2, that is, n >= 4,
  //can we go through the following loop to test
  //whether n has a non-trivial factor or not
  //for (int i = 2; i <= n/2; i++) {
  for (int i = 2; i <= sqrt(n); i++) {
    //it is more efficient to use sqrt(n) than n/2
    //to check the existence of a non-trivial factor of n
    if (n % i == 0) { //i is a factor of n
      return false;
    }
    //ONE non-trivial factor of n is enough to guarantee that n is NOT a prime.
    //else return true; //WRONG!!!!!!!
    //cannot have else return true;
    //when n % i != 0, that is, i is not a factor of n,
    //but some other elements in [2, n/2] may be a factor of n.
    //For example, when n is 9,
    //the possible non-trivial factors are in [2, 9/2], or
    //[2, 4].
    //First check 2, since 9 % 2 != 0,
    //then 2 is not a factor of 9.
    //We cannot return true, which implies that 9 is prime.
    //In fact, 9 is not prime, since it has non-trivial factor 3.
  }

  //if we do not return false inside the loop and leave the function earlier
  //and can reach statements after while-statement,
  //this means none of i in [2, n/2] satisfies n % i == 0.
  //Said differently, none of these candidates for non-trivial factors of n is an actual non-trivial factor of n.
  //Now we can finally declare that n is prime by returning true.
  //Warning: do not write something like
  //std::cout << n << " is prime".
  //print is NOT return.
  //Also, do not print result to the screen unless a function
```

```
    concentrates on printing.
  return true;
}
```

Why this order?

Declaration: Tells the program, "There is a function named `isPrime` that takes an integer and returns a boolean value."

main function: The entry point. Without the declaration above it, main wouldn't know what function `isPrime` is.

Definition: Provides the actual logic of the function `isPrime`.

Here is another example from google AI.

```
#include <iostream>

//1. Declare the function (Prototype)
void makeSandwich(string ingredient);

//2. Define the main function to use it
int main() {
  std::cout << "Starting lunch order..." << std::endl;

  // Calling the function
  makeSandwich("Turkey and Swiss");

  return 0;
}

//3. Define the function (Implementation)
void makeSandwich(string ingredient) {
  std::cout << "Now making a " << ingredient << " sandwich!" << std::endl
    ;
}
```

3 Can we do this exercise, without making it a function?

- Yes. While you can write the entire program without separate functions, all logic must then be contained within the `main` function.

  The underlying philosophy is simple: the work must be performed somewhere. If a task is not delegated to a user-defined function, the responsibility falls to `main`.

- However, encapsulating code into a function offers a distinct advantage: you define the logic once and can **reuse** (call) it as many times as needed throughout your program.

3.1 Code without a function to return the number of occurrences of a target string in the original string

```
//code link: https://onlinegdb.com/cD4Pdyr6q
```

```
//purpose: find out the number of occurrences of a target string in an original string.

//sample input/output:
//Enter a string to search for: abcbc efg
//Enter a string to be searched: bc
//number of occurrences of "bc" in "abcbc efg": 2
#include <iostream>
#include <string>

int main() {
  std::cout << "Enter a string to search for: ";
  std::string str;
  getline(std::cin, str);

  std::cout << "Enter a string to be searched: ";
  std::string target;
  getline(std::cin, target);

  int n = target.length();
  std::string curr;
  int count = 0;
  for (int i = 0; i <= str.length() -  n; i++) {
    curr = str.substr(i, n);

    if (curr == target) {
      count++;
    }
  }

  std::cout << "number of occurrences of \"" << target << "\" in \""
    << str << "\": " << count << std::endl;

  return 0;
}
```

3.2 Code with a function to return the number of occurrences of a target string in the original string

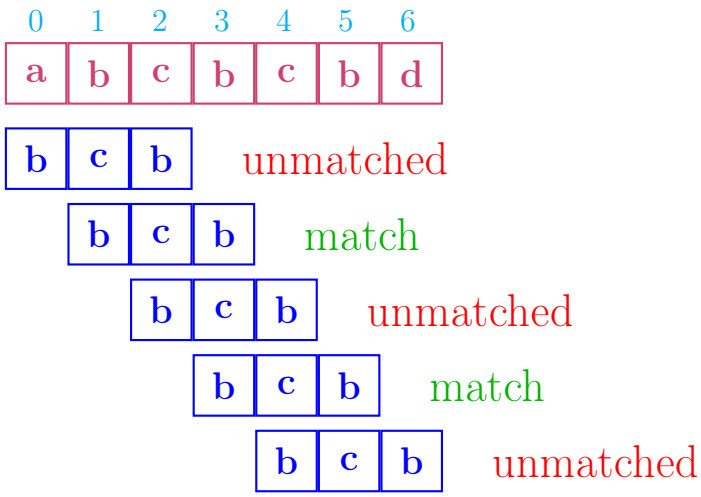
```
//code link: https://onlinegdb.com/lTkK6afQ8
//purpose: find out the number of occurrences of a target string in an original string.

//sample input/output:
//Enter a string to search for: abcbc efg
//Enter a string to be searched: bc
```

```
//number of occurrences of "bc" in "abcbc efg": 2
//number of occurrences of "bcb" in "abcbcbbcbd is 3
#include <iostream>
#include <string>

int num_occurrences( std::string orig, std::string target);

int main() {
  std::cout << "Enter a string to search for: ";
  std::string original;
  getline(std::cin, original);

  std::cout << "Enter a string to be searched: ";
  std::string target;
  getline(std::cin, target);

  int count = num_occurrences(original, target);

  std::cout << "number of occurrences of \""
    << target << "\" in \""
    << original << "\": " << count << std::endl;

  std::string original2 = "abcbcbbcbd";
  std::string target2 = "bcb";
  std::cout << "number of occurrences of \""
    << target2 << "\" in \""
    << original2 << "\" is " <<  num_occurrences(original2, target2) <<
       std::endl;

  return 0;
}

int num_occurrences( std::string orig, std::string target) {
  int n = target.length(); //number of characters in target

  int count = 0;
  for (int i = 0; i <= orig.length() - n; i++) {
    std::string cut = orig.substr(i, n);

    if (cut == target) {
      count++;
    }
  }

  return count;
}
```

4 I wasn't able to understand this problem, even though Ms. Yi wrote the solution.

You means the problem to find out the number of occurrences of a target string in the original string?

Find # of appearances of `"bcb"` in `"abcbcbd"`.



1. Let m be the length of the original string (aka, the string to be searched). For the example, if the original string is "abcbcdb", then m is 7.

2. Let n be the length of the target string (aka, the string to search). For example, if the target string is "bcb", then its length is 3.

3. Starting from index 0, make a substring from the original string with length n.

   If this substring equals the target, increase the number of occurrences by 1.

   Continue until we reach $m - n$, the last index where a substring with n characters can be exracted to get a substring with length n.

Here is a complete code.

```
//code link: https://onlinegdb.com/lTkK6afQ8
//purpose: find out the number of occurrences of a target string in an
   original string.

//sample input/output:
//Enter a string to search for: abcbc efg
//Enter a string to be searched: bc
//number of occurrences of "bc" in "abcbc efg": 2
//number of occurrences of "bcb" in "abcbcbbcbd is 3
#include <iostream>
#include <string>

int num_occurrences( std::string orig, std::string target);

int main() {
  std::cout << "Enter a string to search for: ";
```

```
  std::string original;
  getline(std::cin, original);

  std::cout << "Enter a string to be searched: ";
  std::string target;
  getline(std::cin, target);

  int count = num_occurrences(original, target);

  std::cout << "number of occurrences of \""
    << target << "\" in \""
    << original << "\": " << count << std::endl;

  std::string original2 = "abcbcbbcbd";
  std::string target2 = "bcb";
  std::cout << "number of occurrences of \""
    << target2 << "\" in \""
    << original2 << "\" is " <<  num_occurrences(original2, target2) <<
       std::endl;

  return 0;
}

int num_occurrences( std::string orig, std::string target) {
  int n = target.length(); //number of characters in target

  int count = 0;
  for (int i = 0; i <= orig.length() - n; i++) {
    std::string cut = orig.substr(i, n);

    if (cut == target) {
      count++;
    }
  }

  return count;
}
```

If you still have questions, please read the slides on March 9, 2026, if it is not clear, come to my office hours or ask help from a tutor.

5 I am not fully understanding what I should do for functions when we have to find a value from the length.

Please see the explanation in Section 4.

6 In the old midterms there were a lot of questions about void. Will we have those questions in the midterm?

The problems and testing topics will be similar, but not exactly the same.

7 Can you give us some practice questions that cover most of the topics we have done so far?

Please practice on the old midterms.

8 Where is project 1E?

No project 1E. It was misplaced in gradescope and I took it off now.

9 Are classes and file input on the midterm?

Classes like string will be tested in the midterm. But no file input.

10 What will the midterm exam cover and not cover?

Please read `https://tong-yee.github.io/135/s26/midterm/midterm_review_s26_color.pdf` (colored copy) or `https://tong-yee.github.io/135/s26/midterm/midterm_review_s26_bw.pdf` (black and white copy).

11 When will we review for the midterm?

I gave a review sheet on March 9, 2026. I can cover more on March 11, 2026.

12 This wasn't hard but not using namespace std makes it difficult honestly, the code in c++ is extremely difficult.

If you do not like to use std, then you can wring
`using namespace std;`
Then use `cout, cin, string, setw, boolalpha`.

13 What version of C++ are we using?

We use C++ 11, but your code should work with all versions of C++.